The Unsupported Path: How Windows 11's Setup Stack Undermines Its Own Security Guarantees

Researcher: 0xbekoo

Release Date: 05-21-2026

Table of Contents

Executive Summary

Windows 11's security guarantees rest on two hardware requirements: Secure Boot and TPM 2.0. This report documents that the binaries responsible for enforcing those requirements have not meaningfully changed since Windows 10 — and that the enforcement gap they create is not theoretical. It is in production, at scale, and invisible to the users it affects.

The same binary that bootstraps the entire Windows 11 setup process — setupprep.exe — loads a DLL from a user-controlled directory with no path validation, no signature verification, and no integrity check. In addition, **a security vulnerability was discovered in setup.exe. A function within setupprep.exe loads reservemanager.dll from the sources directory with administrator privileges without proper validation. Using this method, we were able to load our own DLL (see Chapter 4 Unsafe DLL Loading in setupprep.exe)**. The finding does not represent a standalone remote attack vector — triggering it requires write access to installation media. But that is not the point.

This is not a CVE submission. There is no memory corruption, no remote code execution, no novel exploit technique. What this report documents is something harder to score and easier to overlook: a setup stack that publicly declares Secure Boot and TPM 2.0 as mandatory, while silently retaining every mechanism that makes them optional.

The point is what it completes. Across every binary in the Windows 11 setup stack, the same question produces the same answer. winsetup.dll ships a lab bypass that was never removed. setupcompat.dll soft-checks the default installation path. setupprep.exe loads unsigned code without verification. These are not isolated oversights. They are a pattern — and the pattern has a name: a setup stack that was carried forward from Windows 10 into Windows 11 without being held to the security standard Windows 11 publicly claims.

The findings are the result of static analysis of four binaries: **winsetup.dll**, **setupcompat.dll**, **setupprep.exe**, and **winload.efi**. Each was analyzed independently. The findings connect into a single chain — a system installed through LabConfig enters a permanently degraded security state that compounds across every subsequent boot, with no indication to the user that anything is wrong.

The research also produced **WhiteLotus**, a UEFI bootkit developed as part of this research. It consists of two components: a Windows-side loader (**WhiteLotusEXE**) that deploys the bootkit to the EFI

System Partition, and a UEFI DXE driver (**WhiteLotusDXE**) that intercepts the Windows boot chain before the kernel loads — patching winload.efi in memory, disabling DSE, and executing unsigned kernel-mode code. It is included not as a proof-of-concept for a CVE, but as a demonstration that the conditions this report documents are executable — not theoretical.

1. The Problem

> *"Windows 11 will stop these type of attacks out-of-the-box because we're using Secure Boot and Trusted Boot, which use both the required UEFI and TPM hardware."*
>
> — Windows 11 Security — Our Hacker-in-Chief Runs Attacks and Shows Solutions

Out-of-the-box. Not "when configured correctly." Not "on supported hardware." By default, as a guaranteed condition of running Windows 11.

LabConfig is a registry path that Windows Setup reads during installation. It was introduced in Windows 10 for internal lab and deployment scenarios — environments where hardware requirements deliberately do not apply. It has never been formally documented as a supported bypass route. It has never been restricted, gated, or removed. It shipped into Windows 11 unchanged, and it ships today.

When **BypassSecureBootCheck** and **BypassTPMCheck** are set to 1 under **HKLM\SYSTEM\Setup\LabConfig**, the hardware verification functions in winsetup.dll return success without performing any hardware check. The mechanism is unconditional. There is no privilege gate. There is no audit trail beyond a setup log written to disk — a log that records, verbatim: "SecureBoot check bypassed in lab environment.". That log string has shipped, unmodified, in every version of winsetup.dll distributed with Windows 11.

This would be a narrow finding if LabConfig were obscure. It is not. It is the single most widely recommended method for installing Windows 11 on hardware that does not meet the official requirements. YouTube tutorials demonstrating this bypass have accumulated views in the millions. The method is covered in major technology outlets, GitHub gists, and communities across dozens of languages. The tool **Rufus** — one of the most widely used USB creation utilities — exposes this bypass as a single checkbox, enabled by default for unsupported hardware.

The users enabling it, in most cases, do not understand what they are disabling.

1.1. What Follows From This

A system installed through LabConfig does not simply lack Secure Boot at installation time. It enters a permanently degraded security state that compounds across every subsequent boot:

- **bootmgfw.efi** sets **BlVsmpSystemPolicy = 2** — a minimal stub. HVCI, Credential Guard, and Kernel DMA Protection do not initialize.
- **winload.efi** skips SRK provisioning entirely — BitLocker cannot bind to the TPM.

- **FvebpEndTpmBindingEx** is called with **skipPcrExtend = 1** — PCR11 is never extended. BitLocker cannot verify the boot environment against TPM-sealed state.
- **AllowUpgradesWithUnsupportedTPMOrCPU** ensures the system continues receiving Windows 11 feature upgrades — indefinitely, without ever satisfying the requirements that were bypassed at installation.

None of this is visible to the user. The system boots. Updates install. Windows 11 runs. The security posture Microsoft described as the defining characteristic of the operating system is absent — silently, permanently, at scale.

The three bypass routes documented in this report — LabConfig, Scenario 4, and AllowUpgradesWithUnsupportedTPMOrCPU — are not independent findings. They are a chain. LabConfig installs the system. Scenario 4 soft-checks the default path. AllowUpgradesWithUnsupportedTPMOrCPU keeps it alive. The full analysis is in Chapter 3.

2. The Mechanism

HKLM\SYSTEM\Setup\LabConfig is a registry path that Windows Setup reads during installation. When specific DWORD values are present and set to 1, the corresponding hardware check is skipped. This mechanism was introduced in Windows 10 for internal lab and deployment scenarios. It was never removed, restricted, or formally acknowledged as a supported bypass route for Windows 11:

Registry Value	Effect When Set to 1
BypassSecureBootCheck	Skips the Secure Boot requirement check
BypassTPMCheck	Skips the TPM 2.0 requirement check
BypassRAMCheck	Skips the minimum RAM requirement check
BypassStorageCheck	Skips the minimum storage requirement check
BypassCPUCheck	Skips the supported CPU check

In Windows 10, bypassing these checks was not in conflict with any public security statement. In Windows 11, it is. Microsoft explicitly declared Secure Boot and TPM 2.0 as mandatory requirements — not for compatibility reasons, but as the foundation of the operating system's security posture. LabConfig made the transition from Windows 10 to Windows 11 unchanged.

There is no public documentation, no security advisory, no statement in Microsoft Learn or the MSRC blog that acknowledges LabConfig as an accepted bypass route for Windows 11. It simply exists — undocumented at the policy level, while the policy loudly proclaims the opposite

2.1. The Community

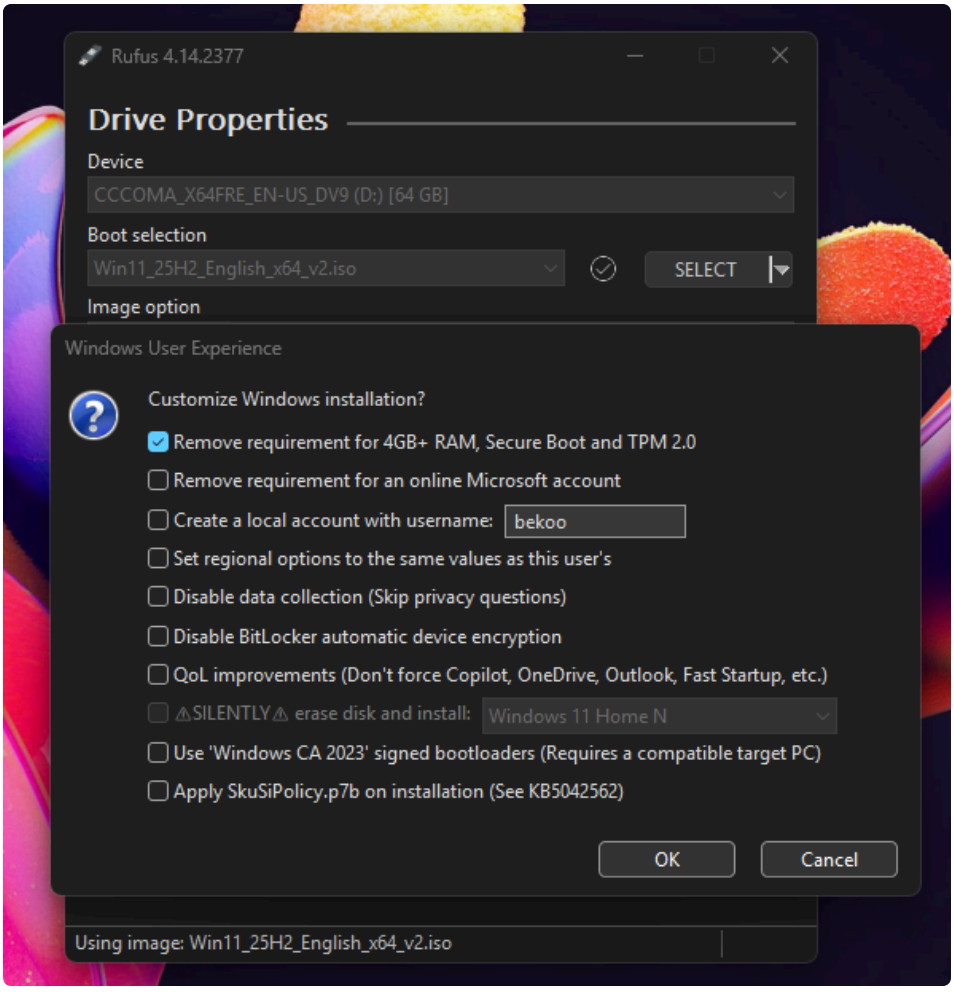
If LabConfig were obscure knowledge, this report would have a different tone. It is not obscure. It is the single most widely recommended method for installing Windows 11 on hardware that does not

meet the official requirements.

YouTube tutorials demonstrating this bypass have accumulated views in the hundreds of thousands and millions. The method is covered in major technology outlets, GitHub gists, and Reddit threads on r/Windows11. On these platforms, installing Windows 11 via LabConfig is described not as a workaround but as the standard route for unsupported hardware.

The quote under "The Problem" is not a hypothetical. It is Microsoft's own description of what Windows 11 is supposed to do. The operative word is out-of-the-box — not "when configured correctly," not "on supported hardware," but by default, as a guaranteed condition of running Windows 11. That guarantee is what LabConfig undermines. Not partially, not in edge cases — systematically, at scale, and invisibly. The bypass is not obscure knowledge confined to security researchers. It is the standard community method for installing Windows 11 on unsupported hardware, documented across forums, YouTube tutorials, and guides in dozens of languages. Without realizing it, many people are currently installing Windows 11 without these features. This is a major weakness for Windows 11.

Bypass methods like LabConfig have become commonplace in communities. This undermines the purpose of the Windows 11 product. As analysis shows, LabConfig is intended for developers. Because Windows marks systems installed via LabConfig as “lab environments”. Here is a photo from the Rufus program:



In fact, using the popular tool Rufus, the Secure Boot and TPM 2.0 requirements for the Windows 11 ISO can be disabled with a single click, and many users enable this without realizing it.

The question this pattern raises is worth stating directly: at what point does a developer-facing bypass mechanism become a security liability when it is normalized as the standard installation route for millions of end users? LabConfig was never designed for this purpose. The log strings embedded in winsetup.dll make that clear. What they do not explain is why a mechanism explicitly labeled "lab environment" continues to ship, unchanged, in a product that publicly guarantees the security properties it bypasses.

The analysis in Chapter 3 provides that answer.

3. Analysis

This chapter contains the analysis for the several modules (winload.efi, winsetup.dll and setupcompat.dll). Each module will be analyzed in a chain.

3.1. Winsetup.dll - Determination of LabConfig

Determining 'LabConfig' parameters has been done by winsetup.dll, so this module is a major player in the problem.

When a user starts to set up the Windows 11, **'ValidateHardwareRequirements'** function checks whether the required features are present. In other words, it is the major player that determines features such as TPM 2.0 or Secure Boot.

```
WDS_EVENT_RESULT ValidateHardwareRequirements(WDS_CALLBACK_DATA
*param_1,WDS_DATA *param_2) {

[...]

BOOLEAN Status = SelectedOSImageIsIotEnterprise();
if (Status == 0) {
    Status = SelectedOSImageIsIotEnterpriseLTSC();
    if (Status != 0) {
            goto CheckRamRequirements; // Secure Boot and TPM checks skipped
    }
    // Normal path: all checks run
    Status = VerifyRAMRequirements(0);
    if (Status != 0 && VerifyProcessorSupported() != 0) {
        if (VerifySecureBoot() != 0) {
            VerifyTPMSupported();
        }
    }
} else {
    // IoT Enterprise: Secure Boot and TPM checks skipped entirely
```

```
    goto CheckRamRequirements;
}
```

When the image type is identified as **IoT Enterprise**, the flow jumps directly to **CheckRamRequirements** — bypassing both **VerifySecureBoot** and **VerifyTPMSupported** entirely. The same applies to **IoT Enterprise LTSC**: if **SelectedOSImageIsIotEnterpriseLTSC()** returns non-zero, execution branches to the same label before either security check is reached.

```
CheckRamRequirements:
Status = VerifyRAMRequirements(Status);
if (iVar1 != 0) {
    iVar1 = VerifyProcessorSupported();
    if (iVar1 != 0) {
    ConstructPartialMsgW(0x400000, "Callback_ValidateHardwareRequirements: HW
Requirements checks passed");
    }
}
```

This function is the single function responsible for verifying every hardware prerequisite that Windows 11 requires before installation proceeds. Every routine it calls is directly tied to this purpose:

- **VerifyRAMRequirements** — enforces the 4 GB minimum
- **VerifyProcessorSupported** — enforces the CPU generation requirement
- **VerifySecureBoot** — enforces the Secure Boot requirement
- **VerifyTPMSupported** — enforces the TPM 2.0 requirement

These are not optional checks. They are the checks. When Microsoft states that Windows 11 requires Secure Boot and TPM 2.0, this function is where that requirement is implemented. There is no other enforcement point in the installation flow.

3.1.1 Bypassing Secure Boot with LabConfig

When the **VerifySecureBoot** is called:

```
BOOLEAN VerifySecureBoot(void)
{
    // Stage 1: Query actual hardware Secure Boot state
    QueryStatus = NtQuerySystemInformation(
        SystemSecureBootInformation,
        &SecureBootInfo,
        sizeof(SecureBootInfo),
        NULL
    );
```

```
    if (NT_SUCCESS(QueryStatus)) {
        // SecureBootInfo._1_1_ = SecureBootCapable field
        IsNotCapable = (SecureBootInfo.SecureBootCapable != TRUE);

        if (IsNotCapable) {
            // Log: "VerifySecureBoot: Not capable of SecureBoot"
            WdsSetupLogMessageW("VerifySecureBoot: Not capable of
SecureBoot");
        } else {
            WdsSetupLogMessageW("VerifySecureBoot: Capable of SecureBoot");
        }

        // If capable → return 1 (TRUE) immediately, skip LabConfig check
        if (!IsNotCapable) {
            return TRUE;
        }

        // Hardware check failed — fall through to Stage 2
    }

    // Stage 2: Hardware check failed or query failed
    BypassValue = RegGetDword(
        HKEY_LOCAL_MACHINE,
        L"System\\Setup\\LabConfig",
        L"BypassSecureBootCheck"
    );

    if (BypassValue != 0) {
        // Log: "VerifySecureBoot: SecureBoot check bypassed in lab
environment"
        WdsSetupLogMessageW(
            "VerifySecureBoot: SecureBoot check bypassed in lab environment");
    }

    // Returns TRUE if bypass is set, FALSE if not
    // Caller cannot distinguish this TRUE from the hardware-verified TRUE
above
    return (BOOLEAN)(BypassValue != 0);
}
```

As you can see, it jumps to the section where LabConfig is checked for the Secure Boot:

```
BypassValue = RegGetDword(
    HKEY_LOCAL_MACHINE,
    L"System\\Setup\\LabConfig",
```

```
    L"BypassSecureBootCheck"
);
```

After the LabConfig check succeeds, the function logs a specific message:

> "*VerifySecureBoot: SecureBoot check bypassed in lab environment*"

The phrase "**lab environment**" is not incidental. It reflects the original intent of this code path — an internal mechanism designed for Microsoft's own testing and validation infrastructure, where hardware requirements deliberately do not apply. In a lab environment, a tester may need to install Windows 11 on a virtual machine, on hardware without TPM, or on a system with Secure Boot disabled for debugging purposes. For those scenarios, LabConfig makes sense.

The question this log message raises is a straightforward one: if this bypass was designed exclusively for lab environments, why is it present in the Windows 11 products?

3.1.2 Bypassing TPM Check with LabConfig

The same situation in the Secure Boot check can be found in others checks. For example, if the TPM cannot found, it checks the LabConfig value:

```
BOOL VerifyTPMSupported() {
[...]

WCHAR SupportState = L"SUPPORTED";
if (!TPMResult) {
    SupportState = L"UNSUPPORTED";
}

if (Status == 0) {
    ConstructPartialMsgW("VerifyTPMSupported: TPM Version %s.",pwVar8);
}
else {
    ConstructPartialMsgW("VerifyTPMSupported:Tbsi_GetDeviceInfo function
failed - 0x%08x",uVar4 & 0xffffffff);
}
```

If the support state of TPM cannot be found, then it searches the LabConfig value:

```
Status =
RegGetDword(0x80000002,L"System\\Setup\\LabConfig",L"BypassTPMCheck");
if (Status != 0) {
    ConstructPartialMsgW("VerifyTPMSupported: TPM check bypassed in lab
```

```
environment");
}
```

Once again it marks the system as 'lab environment'.

3.2. SetupCompat.dll - Persistence of the Unsupported System

Once a system has been installed through LabConfig, a new problem emerges: keeping it there. Windows 11 feature upgrades run their own hardware compatibility checks — independently of the original installation. Without an additional mechanism, a LabConfig-installed system would be blocked from every subsequent upgrade, not because anything changed, but because it never met the requirements to begin with.

3.2.1 Using AllowUpgradesWithUnsupportedTPMOrCPU

It lives in **setupcompat.dll**, and it is called **AllowUpgradesWithUnsupportedTPMOrCPU**. In fact, it was not a planned target of this research, and it was not a key I had encountered before. During the analysis of setupcompat.dll, it surfaced unexpectedly — revealing that it allows unsupported systems to receive Windows 11 feature upgrades. The name implies a narrow scope: CPU and TPM. What the analysis revealed is broader than that.

The name of this registry key is precise and deliberate: it references CPU and TPM, and this key was designed for a specific, documented purpose: allowing systems with unsupported CPUs or older TPM's (instead of TPM 2.0) to receive Windows 11 feature upgrades.

The **IsBypassRegKeySet** function of setupcompat.dll is built around this value. It is called twice inside the DLL, and its sole purpose is to read and return the key's state:

```
Status = RegGetDword( HKEY_LOCAL_MACHINE, L"SYSTEM\\Setup\\MoSetup",
L"AllowUpgradesWithUnsupportedTPMOrCPU" );
if (Status < 0) {
    return FALSE;
}
return (value != 0);
```

Then this return value feeds into **AreHWRequirementsMet** to bypass the requirements.

The AllowUpgradesWithUnsupportedTPMOrCPU key can be used both to bypass restrictions during setup and to install the latest Windows versions on unsupported systems. This enables updates on systems that were installed using LabConfig or another bypass method.

3.2.1.1 Scenario 4 in the Setup

AreHWRequirementsMet function takes three inputs: **ProductType**, **ScenarioStatus**, and **BypassStatus**. Before examining the function itself, both inputs need to be understood.

```
bypassStatus   = IsBypassRegKeySet();
scenarioStatus = GetInstallScenario();

result = AreHWRequirementsMet(productType, scenarioStatus, bypassStatus, ...);
```

IsBypassRegKeySet reads these:

```
HKLM\SYSTEM\Setup\MoSetup
  └─ AllowUpgradesWithUnsupportedTPMOrCPU  (DWORD)
       0  →  bypassStatus = 0  (no bypass)
       1  →  bypassStatus = 1  (bypass requested)
```

And **GetInstallScenario** reads a volatile value written by **setup.exe** at startup:

```
HKLM\SYSTEM\Setup\MoSetup\Volatile
  └─ InstallScenario  (DWORD)
       → This value can be between 1 and 9
```

AreHWRequirementsMet (As a major player) first validates **productType**. Supported values and their corresponding hardware requirement sets are:

productType	Description
1	Client
2	Server
3	IoT Enterprise
4	IoT Enterprise LTSC

Any other value causes the function to return **E_INVALIDARG (0x80070057)** immediately.

Once a valid product type is confirmed, the function selects an evaluation path based on the two inputs:

```
scenarioStatus == 4?
├─ YES  →  EvaluateHardwareRequirementV2() (limited checks – "Running limited
hardware checks.")
└─ NO
```

```
    ├─ bypassStatus != 0?
    │   ├─ YES  →  EvaluateHardwareRequirementV2()  (soft-check - "TPM and
CpuFms checks will be soft-checked.")
    │   └─ NO   →  EvaluateHardwareRequirement()    (full strict evaluation)
```

In code:

```
if (scenarioStatus == 4) {
    // Boot Scenario / Limited Checks
    EvaluateHardwareRequirementV2(requirementSet, filterFlags, &results,
&resultCount);
}
else if (bypassStatus != 0) {
    // Registry bypass - TPM and CpuFms become soft checks
(AllowUpgradesWithUnsupportedTPMOrCPU)
    EvaluateHardwareRequirementV2(requirementSet, filterFlags, &results,
&resultCount);
}
else {
    // Normal - full strict evaluation
    EvaluateHardwareRequirement(requirementSet, filterFlags, &results,
&resultCount);
}
```

EvaluateHardwareRequirementV2 checks the hardware with '**smooth-condition**'. Instead of another function (EvaluateHardwareRequirement), if this function finds the hardware but the version is old, then it will accept the system. Another function (**EvaluateHardwareRequirement**) does the opposite. if the versions don't match, then it won't accept the system.

As we can see from the code, Scenario 4 checks the system with 'smooth-conditions'.

We can track this Scenario values. This scenario value is used by **setupprep.exe** itself. In **CreateCommandLine** function, it determines the Command Argument with the value:

```
CommandArgument = *(int *)(param_1 + 0x74);
if (CommandArgument == 1) {
    ScenarioValue = L"/Media %s";
}
else if (CommandArgument == 2) {
    ScenarioValue = L"/Recovery %s";
}
else if (CommandArgument == 4) {
    ScenarioValue = L"/Boot %s";
}
else if (CommandArgument == 5) {
```

```
    ScenarioValue = L"/Update %s";
}
else if (CommandArgument == 6) {
    ScenarioValue = L"/Package %s";
}
else if (CommandArgument == 7) {
    ScenarioValue = L"/Web %s";
}
else if (CommandArgument == 8) {
    ScenarioValue = L"/AzureHost %s";
}
else {
    if (CommandArgument != 9) {
        goto Exit;
    }
    ScenarioValue = L"/Servicing %s";
}
```

The fourth option (Scenario 4) is the blue-screen environment that appears when a user begins installing Windows.

3.2.1.2 OnInvoke — Per-Requirement Loop and Final Override

After **AreHWRequirementsMet** returns, **OnInvoke** iterates over every entry in the result array. For each failed requirement (+0x80 == 0), it checks whether a per-requirement override is possible.

These evaluate functions populate an array of **HWREQCHK_DEVICE_HARDWARE_EVALUATION** structs (0x84 bytes each). The field at offset `+0x80` in each entry is the per-requirement result flag:

```
+0x80 == 0  →  requirement FAILED
+0x80 == 1  →  requirement PASSED
```

This override stage is only reachable when **scenarioStatus == 4**:

```
for (currentRequirement = firstResult; currentRequirement != lastResult;
currentRequirement++) {
    if (currentRequirement->evaluationResult == 0) {
        if (GetInstallScenario() == 4) {
            // Map requirement name to its corresponding bypass key

            if (wcscmp(requirementName, L"Memory") == 0) bypassKey =
L"BypassRAMCheck";
            else if (wcscmp(requirementName, L"UefiSecureBoot") == 0)
bypassKey = L"BypassSecureBootCheck";
            else if (wcscmp(requirementName, L"TpmVersion") == 0) bypassKey =
```

```
L"BypassTPMCheck";
            else goto HardBlock;

            ShouldOverrideCompatCheck(TargetBypassKey, &overrideAllowed);

            if (overrideAllowed) {
                currentRequirement->evaluationResult = 1;
                // mark as PASSED
                // logs: "%s check is bypassed in lab environment."
            }
        }

        if (currentRequirement->evaluationResult == 0) {
            ReportIssue(HardBlock);     // still failed - block the upgrade
        }
    }
    else {
        ReportIssue(NoIssue);
    }
}
```

ShouldOverrideCompatCheck reads a dedicated registry value for each requirement:

```
HKLM\SYSTEM\Setup\LabConfig
  └─ BypassTPMCheck          (DWORD, non-zero = override allowed)
  └─ BypassRAMCheck
  └─ BypassSecureBootCheck
```

Or in code:

```
Status = KeyExists(TargetBypassKey, L"SYSTEM\\Setup\\LabConfig", local_res10);
if (-1 < Status) {
    if (local_res10[0] != 0) {
        local_res10[0] = 0;
        Status = ValueExists(...,L"SYSTEM\\Setup\\LabConfig",
                             TargetBypassKey,local_res10);
        if (Status < 0) {
            goto LAB_1800130d4;
        }
        if (local_res10[0] != 0) {
          Status = GetValue(..., L"SYSTEM\\Setup\\LabConfig",
                            TargetBypassKey,local_res18);
          if (Status < 0) {
              goto LAB_1800130d4;
          }
```

```
            StatusKey = (uint)(local_res18[0] != 0);
          }
        }
        *StatusOutput = StatusKey;
        goto LAB_180013147;
      }
    }
```

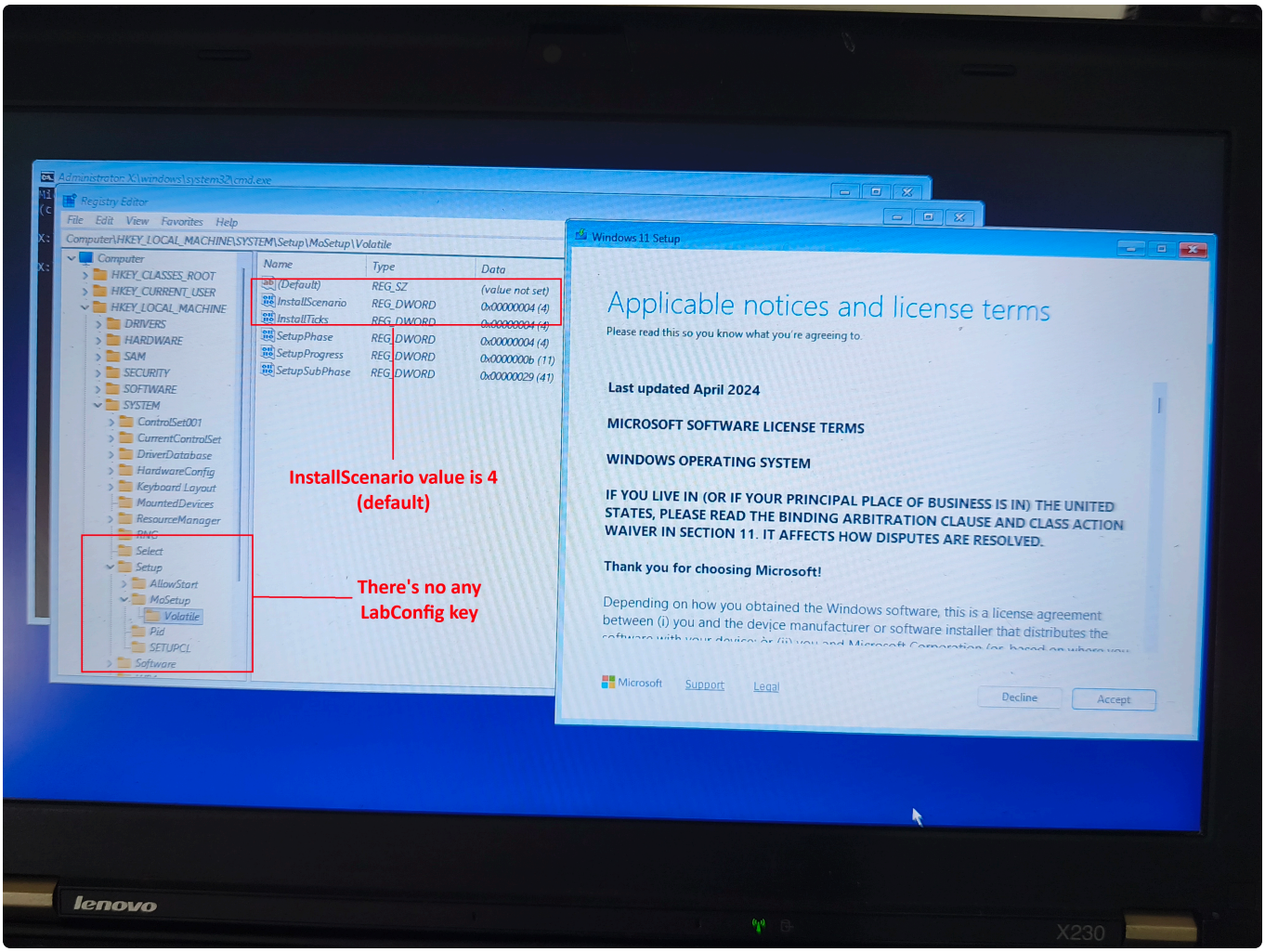
In Scenario 4, the system begins searching for any missing hardware in LabConfig. It attempts to retrieve the data using **GetValue**; if successful, it considers the hardware to be bypassed and proceeds with the steps.

3.2.2 Testing to Bypass Setup and Upgrade Progress in X230

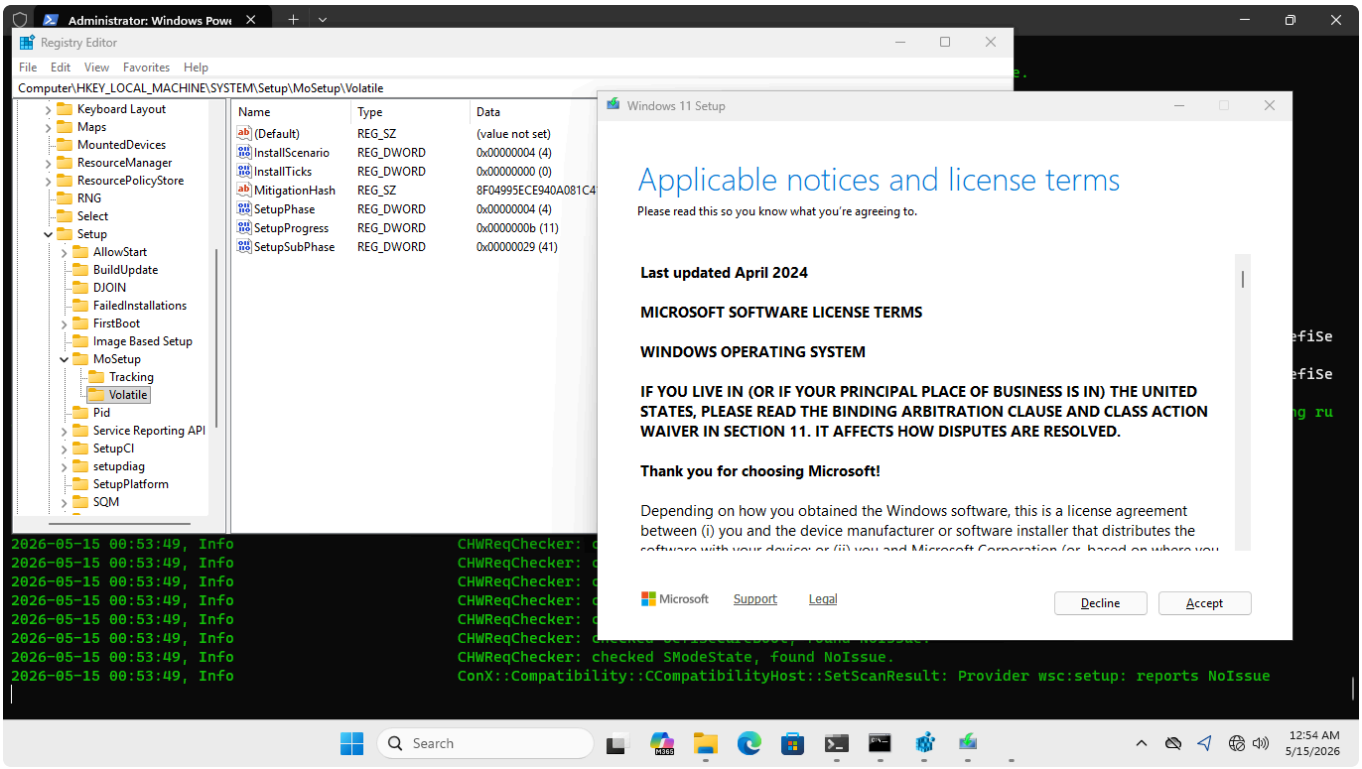
In this chapter, i will show my analyses with my old laptop X230.

3.2.2.1 Scenario 4 in Setup Progress

From what I can see in the analysis, the InstallScenario value is set 4 in normal scenario. Remember it from the analyses that it checks the hardware with 'smooth-check':



Also we can check the logs, if we run **setup.exe /boot**:



Here's the logs:

```
IsBypassRegKeySet: Error reading bypass registry key. HRESULT = 0x80070002.
IsBypassRegKeySet: Unsupported CPU and TPM check is bypassed through regkey: FALSE
GetInstallScenario: Install scenario: 4
X   hwreqchk: TRACE,Windows::Compat::HardwareRequirements::HardwareInventory::QueryHardware,117,UefiSe

X   hwreqchk: TRACE,Windows::Compat::HardwareRequirements::HardwareInventory::QueryHardware,118,UefiSe

X   hwreqchk: TRACE,Windows::Compat::HardwareRequirements::Evaluate::PreProcessRules,345,*** Adding ru
' to inventory list ***
CHWReqChecker: checked SSE2Support, found NoIssue.
CHWReqChecker: checked NXSupport, found NoIssue.
CHWReqChecker: checked CompareExchange128Support, found NoIssue.
CHWReqChecker: checked LahfSahfSupport, found NoIssue.
CHWReqChecker: checked PrefetchWSupport, found NoIssue.
CHWReqChecker: checked PopCntSupport, found NoIssue.
CHWReqChecker: checked SSE4_2ProcessorSupport, found NoIssue.
CHWReqChecker: checked CpuCores, found NoIssue.
CHWReqChecker: checked TpmVersion, found NoIssue.
CHWReqChecker: checked Memory, found NoIssue.
CHWReqChecker: checked UefiSecureBoot, found NoIssue.
CHWReqChecker: checked SModeState, found NoIssue.
ConX::Compatibility::CCompatibilityHost::SetScanResult: Provider wsc:setup: reports NoIssue
```

The reason is simple. The setup.exe of Windows 11 does not check the TPM version **for Scenario 4. Although Microsoft states that TPM 2.0 is required for the Windows 11 itself**, the TPM version on the system is not checked at all—only its presence is verified.

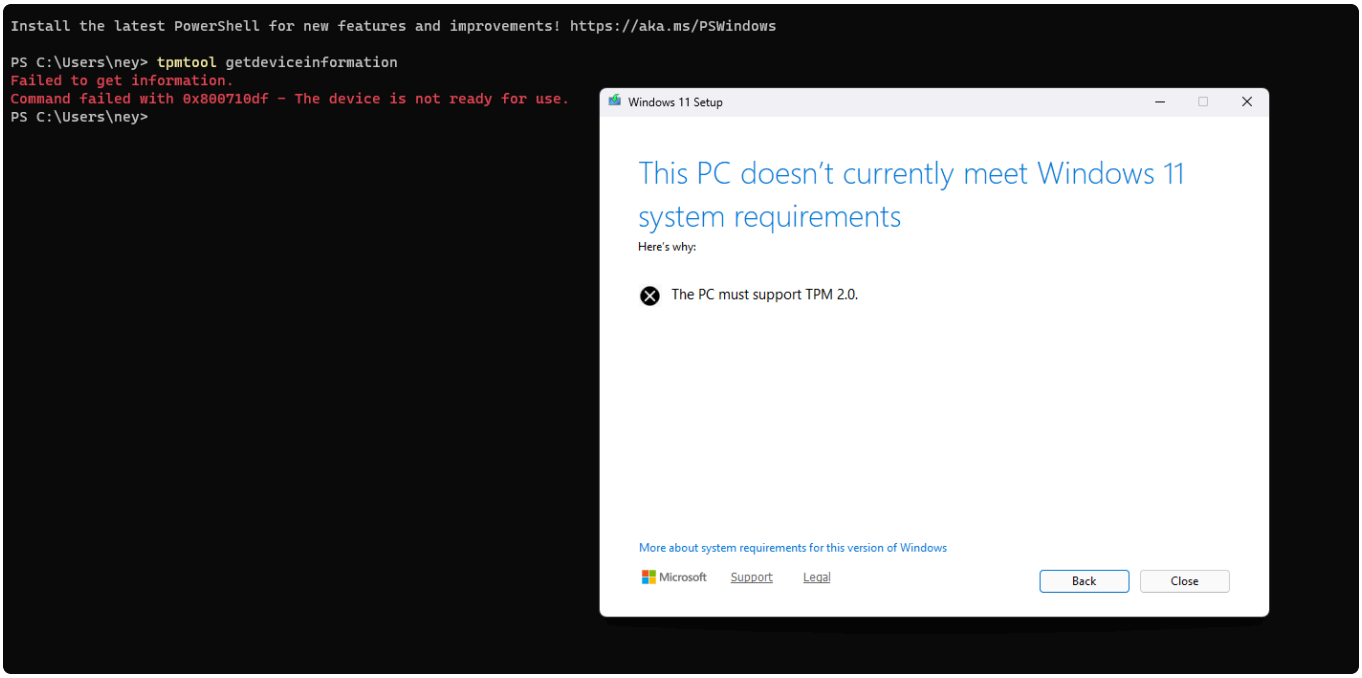
Focus on the "**NoIssue**" text in the log. It checks the hardware, but even if they are insufficient, it accepts them simply because they are present in the system. This completely bypasses requirements like TPM 2.0 that Windows 11 looks for, because the version check is not performed.

We can verify this in X230 and VMware:

```
2026-05-15 01:22:14, Info                                       Co
PS C:\Users\bekoo> tpmtool getdeviceinformation

-TPM Present: True
-TPM Version: 1.2
-TPM Manufacturer ID: STM
-TPM Manufacturer Full Name: ST Microelectronics
-TPM Manufacturer Version: 13.8
-PPI Version: 1.2
```

This is the TPM version of my X230 laptop. And if we remove TPM in Windows 11 VM and run **setup.exe /boot**, we will see an error for TPM:



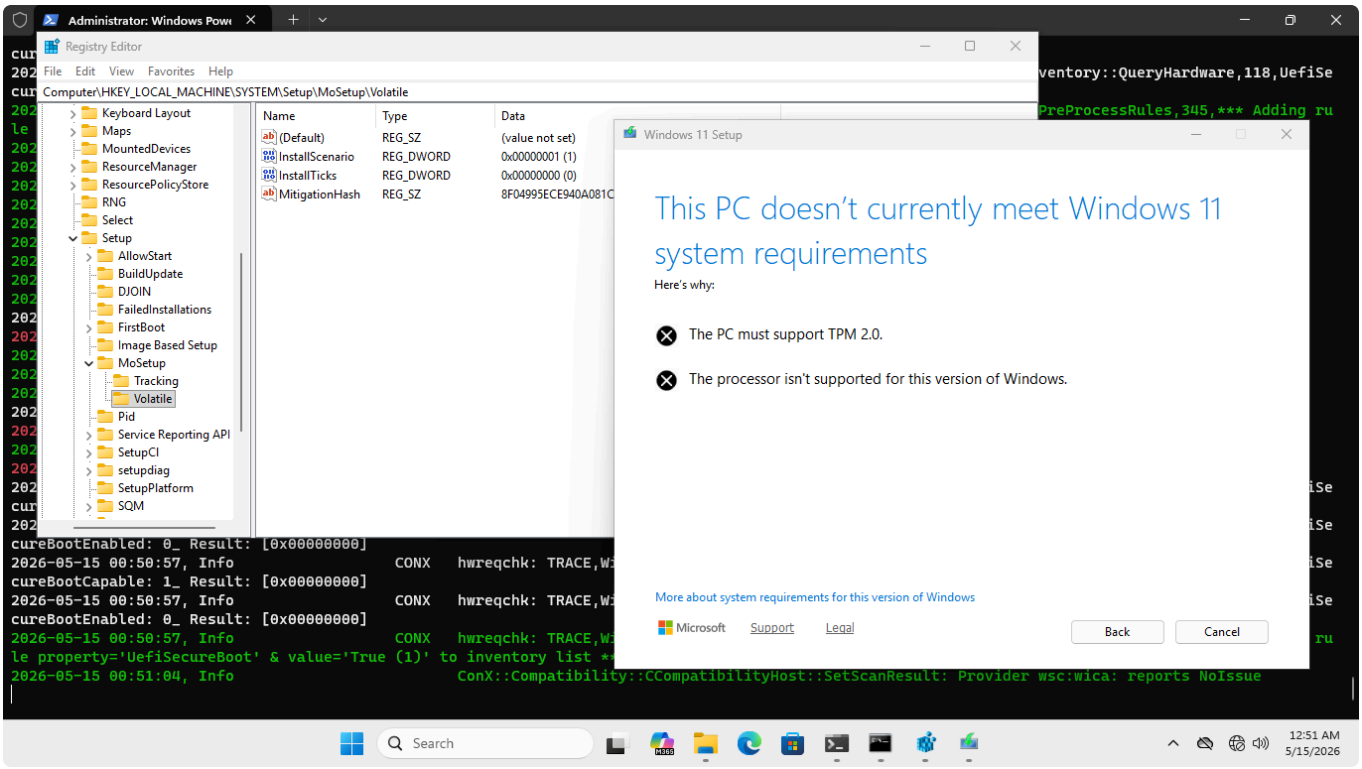
For scenario 4 (default option), **setup.exe** simply checks whether the system has a TPM. It does not perform any version checks.

In addition, this **Scenario 4** value can be also for the **/auto upgrade** code, because windows does not protect the value and the user can change the value of the key. So that unsupported systems can be upgraded with this technique (or using **AllowUpgradesWithUnsupportedTPMOrCPU**). **And these methods can help keep older systems that aren't supported by Windows 11 up to date!**

3.2.2.2 Scenario 1 in Setup Progress

As we can see from the analyses, Scenario 1 is belong to **upgrade progress** (/auto upgrade) in normal scenario, but in this report, I wanted to change the InstallScenario value to 1 under the /boot option to show the difference from Scenario 4 and include the result.

InstallScenario is not protected, so we can change this. After the running **setup.exe /boot**, i changed the value to 1 and saw this:



This is the check that needs to be done in real-world scenarios. If the installation scenario is not Scenario 4, it performs all the necessary checks. The X230 system (as we saw earlier) has a TPM 1.2, and its processor does not support Windows 11, as reported by setup.exe

Also we can check the logs:

```
CHWReqChecker: checked SSE2Support, found NoIssue.
CHWReqChecker: checked NXSupport, found NoIssue.
CHWReqChecker: checked CompareExchange128Support, found NoIssue.
CHWReqChecker: checked LahfSahfSupport, found NoIssue.
CHWReqChecker: checked PrefetchWSupport, found NoIssue.
CHWReqChecker: checked PopCntSupport, found NoIssue.
CHWReqChecker: checked SSE4_2ProcessorSupport, found NoIssue.
CHWReqChecker: checked CpuCores, found NoIssue.
CHWReqChecker: checked TpmVersion, found NoIssue.
GetInstallScenario: Install scenario: 1
CHWReqChecker: checked TpmVersion, found HardBlock.
CHWReqChecker: checked Memory, found NoIssue.
CHWReqChecker: checked UefiSecureBoot, found NoIssue.
CHWReqChecker: checked SystemDiskSize, found NoIssue.
GetInstallScenario: Install scenario: 1
CHWReqChecker: checked CpuFms, found HardBlock.
CHWReqChecker: checked SModeState, found NoIssue.
ConX::Compatibility::CCompatibilityHost::SetScanResult: Provider wsc:setup: reports HardBlock
hwreqchk: TRACE,Windows::Compat::HardwareRequirements::HardwareInventory::QueryHardware,117,Uef
```

It logs the hardwares as '**HardBlock**'. **However, the exact opposite happens during a Windows installation.**

So we can say that Microsoft hasn't taken many precautions regarding Windows 11 requirements in setup.exe. It seems that content from Windows 10 has been included in Windows 11 without being updated.

3.3. Winload.efi — Boot-Time Security Degradation

The findings in Chapters 3.1 and 3.2 describe how a system arrives at a non-compliant state. This chapter describes what that state means on every boot that follows.

winload.efi is not part of the setup stack. It has no knowledge of how the system was installed, which registry keys were set, or which checks were bypassed. It operates on what it finds at runtime: the Secure Boot state reported by firmware, the TPM version detected by the TCG protocol stack, and the boot environment as it exists at the moment of execution. On a system installed through LabConfig, what it finds is always the same — Secure Boot absent, TPM version insufficient or absent, no authenticated VBS policy. winload.efi responds to these conditions exactly as it was designed to, by degrading gracefully. The problem is not that winload.efi behaves incorrectly. The problem is that its correct behavior, applied to a LabConfig-installed system, permanently disables the security features Windows 11 is built around.

winload.efi is responsible for initializing the security environment that the kernel inherits: assembling the VSM policy, reporting the TPM state, provisioning the Storage Root Key, and establishing the BitLocker TPM binding. Each of these operations is documented below. Each of them fails, silently and permanently, on a system where Secure Boot is absent.

The analysis covers four functions. All four were verified by static analysis of winload.efi.

3.3.1. VBS Policy — Secure Boot State Determines Whether HVCI and Credential Guard Initialize

The function that determines the system-wide VBS security posture is **BlVsmSetSystemPolicy** (0x1801bdf7c). It constructs **BlVsmpSystemPolicy** — the global value that controls whether HVCI, Kernel DMA Protection, Credential Guard, and Secure Launch will initialize. Its behavior is entirely conditional on the Secure Boot state it finds at boot time.

The function begins by querying Secure Boot state through **SbIsEnabled2**:

```
// Inside BlVsmSetSystemPolicy @ 0x1801bdf7c

int  sbQueryResult = SbIsEnabled2(1, local_res10);
char sbEnabled     = local_res10[0];
if (sbQueryResult < 0) {
    sbEnabled = '\0';
}
```

When Secure Boot is active (sbEnabled != 0), the function reads the **VbsPolicy** UEFI NVRAM variable from the Secure Boot private namespace, evaluates it against hardware capabilities, and writes the result back — establishing an authenticated, tamper-evident policy:

```
// Secure Boot active path
int vbsPolicyResult = BlSecureBootGetBootPrivateVariable(
    L"VbsPolicy",
    &vbsPolicyData,
    &vbsPolicySize
);

// Policy bits assembled from hardware capabilities:
// bit 0x600 → HVCI + Kernel DMA Protection
// bit 0x080 → Secure Launch (DRTM)
// bit 0x040 → Credential Guard

BlSecureBootEFISetVariable(L"VbsPolicy", assembledPolicy, 3, 8, &policyBlock);
```

When Secure Boot is disabled (sbEnabled == 0), the function takes a different branch:

```
// Secure Boot disabled path
if (sbEnabled == '\0') {
    BlVsmpSystemPolicy = 2;    // hard-coded minimal stub
    return 0;                  // function exits — nothing else runs
}
```

BlVsmpSystemPolicy = 2 is a minimal stub value. It signals to the rest of the loader that VBS is structurally present, but no real policy was loaded. The **VbsPolicy** NVRAM variable is never consulted, never verified, and never updated. HVCI, Kernel DMA Protection, Secure Launch, and Credential Guard will not initialize — not because the hardware lacks support for them, but because the policy that would have enabled them was never assembled.

A system installed via LabConfig produces BlVsmpSystemPolicy = 2 on every subsequent boot, permanently. This is not a degraded state that resolves over time or after updates. It is the permanent condition of every LabConfig-installed system.

3.3.2. VSM Loader Block — How winload.efi Reports the Security Environment to the Kernel

The VSM Loader Block is the data structure winload.efi constructs and passes to the kernel to describe the boot-time security environment. Two functions populate the fields relevant to this analysis.

OslpVsmInitializeLoaderBlock (0x180026950) assembles a flags field at offset **+0x5a0** of **OslVsmLoaderBlock**. Bit 0x10 of this field marks the boot environment as Secure Boot authenticated:

```
// Inside OslpVsmInitializeLoaderBlock @ 0x180026950

int  sbQueryResult    = SbIsEnabled2(0, &local_res10);
```

```
char sbActive         = (char)local_res10;
uint authenticatedFlag = 0;

if ((sbQueryResult >= 0) && (sbActive != '\0')) {
    int  testRootResult   = SbIsTestRootTrusted(&local_res18);
    char testRootBlocked  = '\0';
    if (testRootResult < 0 || local_res18[0] == '\0') {
        testRootBlocked = SbIsTestSigningBlocked();
    }
    if (testRootBlocked == '\0') {
        authenticatedFlag = 0x10;   // authenticated flag SET
    }
}

*(uint*)((longlong)OslVsmLoaderBlock + 0x5a0) |= authenticatedFlag;
```

On a LabConfig-installed system,**sbActive = 0**, **authenticatedFlag* remains zero, and the `0x10` bit is never written. The same field at *+0x5a0* carries the following bits, all of which remain zero on a LabConfig system:

Bit	Meaning
0x0010	Secure Boot authenticated
0x0040	TPM-bound LKey present
0x0080	DRTM / Secure Launch active
0x4000	Kernel DMA Protection active
0x8000	VSM LKey provisioned

OslpVsmInitializeSecureCoreInfo (0x1800267e4) encodes the TPM version detected during boot into OslVsmLoaderBlock at offset **+0x744**, passing it to the kernel as **SecureCoreInfo.TpmVersion**:

```
// Inside OslpVsmInitializeSecureCoreInfo @ 0x1800267e4

// DAT_180339fc8: 0 = no TPM, 1 = TPM 1.2, 2 = TPM 2.0
*(uint*)((longlong)OslVsmLoaderBlock + 0x744) =
    (uint)(byte)tpmVersion << 8
    | sbAuthFlags
    | -(uint)(lkeyBoundFlag != '\0') & 0x40;
```

tpmVersion (DAT_180339fc8) is populated by **BlpTpmInitialize**, which probes the firmware for TCG protocol implementations:

```
// Inside BlpTpmInitialize @ 0x1801ab064

int tpm20Result = BlTpmpAcquireProtocol(&BlTpmpTcg20Implementation);
if (tpm20Result < 0) {
    BlTpmpAcquireProtocol(&BlTpmpTcg12Implementation);
}
// tpmVersion = 0 → no TPM
// tpmVersion = 1 → TPM 1.2
// tpmVersion = 2 → TPM 2.0
```

The kernel-visible TPM version field on each system type:

System	tpmVersion	OslVsmLoaderBlock+0x744	tpm.msc result
X230 (TPM 1.2, LabConfig)	1	0x0100	Inactive / Not manageable
VMware (no TPM, LabConfig)	0	0x0000	Not found
Clean install (TPM 2.0, SB active)	2	0x0200	Ready

This is the value **tpm.msc** surfaces, and it explains why TPM appears inactive on LabConfig-installed systems regardless of what hardware is physically present.

3.3.3. SRK Provisioning — Why BitLocker TPM Binding Is Impossible

The Storage Root Key (SRK) is the TPM 2.0 primary key under which all other security keys — BitLocker Volume Master Keys, Windows Hello credentials, and VBS LKeys — are sealed. Without it, none of those keys can be created or accessed.

OslpProvisionTpmSrk (0x1800392a4) is responsible for creating the SRK during boot. It begins with an unconditional version guard:

```
// Inside OslpProvisionTpmSrk @ 0x1800392a4

void OslpProvisionTpmSrk(void) {
    if (tpmVersion != 2) {
        return;   // Not TPM 2.0 — silent return, no ETW event
    }

    int  aes256Result  = TpmApiTestAes256Capability20(0);
    char aes256Capable = (aes256Result >> 0x1f & 1U) + 1;
```

```
    int  rsa3kResult  = TpmApiTestRsa3kCapability20(0);
    char rsa3kCapable = (rsa3kResult >> 0x1f & 1U) + 1;

    int srkHandle = 0;
    TpmW8ApiCreateSrk20(
        (HTPMCONTEXT__*)0x0,
        &srkHandle,
        (uint)(aes256Capable == 1),
        (uint)(rsa3kCapable  == 1)
    );
}
```

If tpmVersion != 2, the function returns immediately. There is no fallback, no warning, and no **BOOT_TPM_SRK_PROVISION_STATUS** TW event. The SRK does not exist.

System	tpmVersion	SRK Created	BitLocker TPM Binding	tpm.msc
X230 (TPM 1.2, LabConfig)	1	No	No	Inactive
VMware (no TPM, LabConfig)	0	No	No	Not found
Clean install (TPM 2.0, SB active)	2	Yes	Active	Ready

LabConfig permitted setup.exe to skip the TPM version check at installation time. winload.efi enforces TPM 2.0 for SRK provisioning unconditionally — it has no knowledge that the system was installed under lab conditions. The result is a system where installation succeeded, but the TPM security chain is permanently broken from the first post-installation boot onward.

3.3.4. TPM Binding Termination — BitLocker Loses Its PCR11 Measurement

All BitLocker-related TPM operations in winload.efi are gated by a bitmask state variable **fveStateBitmask** (DAT_180352f20). The function **FvebpCanUseTpm** (0x18004e42c) is the gatekeeper for all downstream TPM operations:

```
// Inside FvebpCanUseTpm @ 0x18004e42c

uint FvebpCanUseTpm(void) {
    if (
        (fveStateBitmask >> 9  & 1) != 0 &&    // bit 9:  TPM initialized
        (fveStateBitmask >> 10 & 1) == 0 &&    // bit 10: no TPM hardware
error
```

```
        (fveStateBitmask >> 11 & 1) == 0 &&    // bit 11: TPM not reported
absent
        (fveStateBitmask >> 12 & 1) == 0 &&    // bit 12: binding not
explicitly disabled
        (fveStateBitmask >> 13 & 1) == 0       // bit 13: binding not force-
ended
    ) {
        return 1;   // TPM usable
    }
    return 0;
}
```

When Secure Boot is disabled, **BlFveSecureBootCheckpointInsecure** (`0x18004ba48`) is called in place of **BlFveSecureBootCheckpointBootApp**. This function marks the boot as insecure and calls **FvebpEndTpmBindingEx** with `skipPcrExtend = 1`:

```
// Inside BlFveSecureBootCheckpointInsecure @ 0x18004ba48

int BlFveSecureBootCheckpointInsecure(char param_1) {
    if (param_1 != '\0') {
        fveStateBitmask |= 0x40000;   // insecure boot flag
    }
    FvebpEndTpmBindingEx(
        -(uint)(param_1 != '\0') + 2,   // 2 = insecure-end
        1                               // skipPcrExtend = 1
    );
}
```

Inside **FvebpEndTpmBindingEx**, the skipPcrExtend parameter determines whether PCR11 is extended:

```
// Inside FvebpEndTpmBindingEx @ 0x18004d440

// Secure Boot active path (skipPcrExtend = 0):
SipaCheckBitLockerPcrCapped();
SipaLogBitLockerCap();
if (tpmVersion == 1) {
    Tpm12ApiExtendPCR(&FvebGlobalData, &pcr11ExtendData);
} else if (tpmVersion == 2) {
    TpmApiExtendPCR20(&FvebGlobalData, 0xb, &pcr11ExtendData);
}

// Secure Boot disabled path (skipPcrExtend = 1):
// PCR extension calls are never reached.
```

```
// Both paths:
fveStateBitmask |= 0x2000;   // binding "completed" flag — set regardless
```

The binding completion flag **0x2000** is set in both the secure and insecure paths. Any subsequent caller that reads *fveStateBitmask* sees a completed binding. When **skipPcrExtend = 1**, however, PCR11 is never extended — the TPM binding record is closed without a valid measurement. BitLocker cannot verify the boot environment against TPM-sealed state.

The result is a system that reports a completed TPM binding, while that binding carries no measurement of the actual boot environment. The deception is not visible to the user, and it is not surfaced by any standard diagnostic tool.

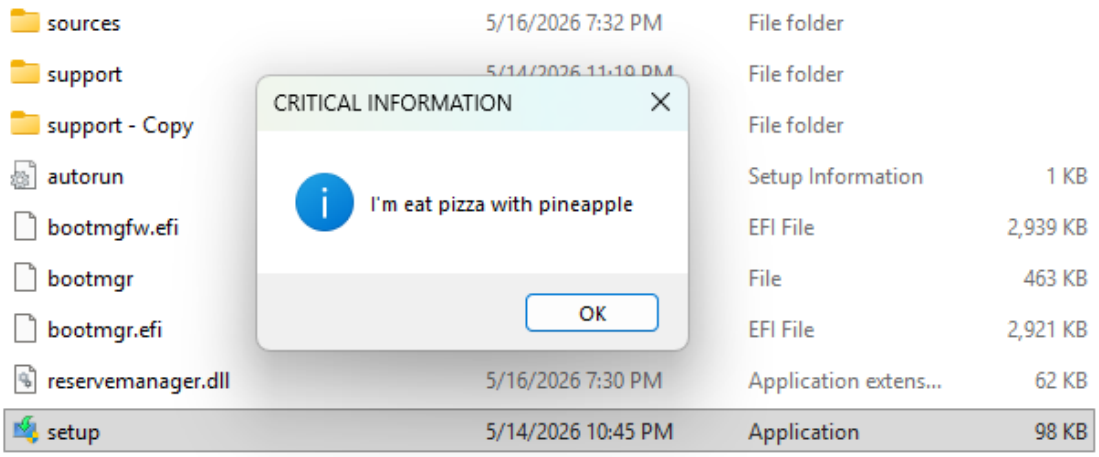
4. Unsafe DLL Loading in setupprep.exe

The same setup stack that skips hardware requirement checks in winsetup.dll and soft-checks Secure Boot in setupcompat.dll also fails a basic DLL loading security control in setupprep.exe. The finding does not yield a practical attack surface in real-world scenarios — the binary is delivered as part of the Windows 11 installation media, and triggering the behavior requires the ability to modify the contents of that media. It is included in this report not as a vulnerability submission, but as an additional data point: the binary responsible for bootstrapping the entire setup process loads a DLL from a user-controlled directory with no path validation, no signature verification, and no integrity check.

The behavior is located in the **LoadReserveMgrModule** function:

```
Status = CombinePath(Pathdirectory, L"ReserveManager.dll", 0x0, &local_res8);
if (Status < 0) {
    CheckToBreakOnFailure(Status);
    LibraryAddr = 0x0;
} else {
    LibraryAddr = LoadLibraryExW(local_res8, (HANDLE)0x0, 0);
    if (LibraryAddr == 0x0) {
        Status = GetLastError();
        goto exit;
    }
}
```

The function constructs a path from the setup directory and immediately loads whatever DLL is found at that location. No flags are passed to **LoadLibraryExW** that would restrict loading to a known-safe path or require a signed binary. A DLL placed in the **sources** directory of a mounted or extracted ISO as *ReserveManager.dll* is loaded and executed with the same privileges as setup.exe, which runs as administrator.



5. PoC: WhiteLotus — A UEFI Bootkit

The preceding analysis documents a setup and boot stack that produces non-compliant systems at scale — silently, permanently, and without any user-visible indication. WhiteLotus was developed to answer a practical question: what does an attacker do with this environment?

The answer is straightforward. On a system where Secure Boot is absent, the boot chain offers no resistance. Every integrity guarantee that Windows 11 describes as foundational — verified boot, DSE enforcement, VBS initialization — depends on Secure Boot being present at the moment winload.efi executes. Remove it, and those guarantees do not degrade. They disappear.

WhiteLotus is a UEFI bootkit that operates in exactly this environment. It is not a demonstration of a novel exploit technique. It does not bypass Secure Boot — it does not need to. It runs in the space that LabConfig creates.

5.1. Components

WhiteLotus consists of two components:

WhiteLotusDXE is a UEFI DXE driver that performs all boot-time operations. It installs a hook on **EFI_BOOT_SERVICES->LoadImage()** at driver entry. Every EFI binary loaded during boot passes through this hook. When bootmgfw.efi is detected, WhiteLotusDXE hooks **ImgArchStartBootApplication** — the function bootmgfw.efi calls to transfer execution to winload.efi. This hook fires exactly once, at the moment winload.efi is about to receive control. Inside the hook, WhiteLotusDXE locates winload.efi in memory by scanning for its PE header and hooks **OslFwpKernelSetupPhase1** — the function winload.efi calls immediately before handing control to the kernel. At this point, ntoskrnl.exe is present in memory and has not yet executed. All kernel-level patches are applied here: DSE is disabled, and unsigned kernel-mode code executes.

The interception chain — **LoadImage** → **ImgArchStartBootApplication** → **OslFwpKernelSetupPhase1** — is possible only because Secure Boot is absent. With Secure Boot active, bootmgfw.efi verifies the signature of every EFI binary before execution. WhiteLotusDXE, unsigned, would not load.

WhiteLotusEXE is a Windows-side loader that deploys WhiteLotusDXE to the EFI System Partition and configures the boot environment. It uses the **ICMLuaUtil** COM object — abusing the CMSTPLUA interface's elevated execution path — to run at HIGH integrity level without triggering a UAC prompt. This method is effective on the latest shipping version of Windows 11.

5.2. What This Demonstrates

WhiteLotus does not demonstrate that UEFI bootkits are new. It demonstrates that on a LabConfig-installed system, deploying one requires no Secure Boot bypass, no firmware vulnerability, and no kernel exploit. The only precondition is the absence of Secure Boot — a condition that LabConfig produces by design, at the user's request, on millions of systems.

The capabilities WhiteLotus exercises — pre-kernel code execution, DSE bypass, unsigned driver loading — are not theoretical. They are the direct, observable consequence of the conditions documented in Chapters 3.1 through 3.3. Every system installed through LabConfig is a system where these capabilities are available to any attacker who can write to the EFI System Partition.

5.3. Further Reading

The full technical writeup, source code, and a demonstration video are available at the following links:

- **Blog:** WhiteLotus — Technical Writeup
- **Repository:** GitHub — WhiteLotus
- **Demo Video (Vimeo):** WhiteLotus Demonstration

6. Reviews and Recommendations

The findings in this report are not isolated bugs. They represent a systemic condition: a setup stack that was carried forward from Windows 10 into Windows 11 without being updated to reflect the security guarantees Windows 11 publicly committed to. The recommendations below are written in that context. They are not requests to patch individual functions — they are a request to close the gap between what Windows 11 promises and what it delivers.

Each recommendation is categorized by implementation scope: **immediate** (policy or binary-level changes with limited surface impact) and **long-term** (architectural changes that require broader planning).

Recommendation 1: Remove LabConfig from Production Binaries *(Immediate)*

LabConfig was designed for internal lab and deployment environments. Its presence in the production Windows 11 setup binaries — winsetup.dll and setupcompat.dll — is inconsistent with Windows 11's public security posture.

The specific values that should be removed or restricted in production builds are:

- BypassSecureBootCheck
- BypassTPMCheck
- BypassRAMCheck
- BypassStorageCheck
- BypassCPUCheck

These values serve a legitimate purpose in controlled testing environments. They have no legitimate purpose in a binary that ships to end users. The log string "*SecureBoot check bypassed in lab environment*" — present, unmodified, in every shipping version of winsetup.dll — is itself evidence that this code path was never intended for production use.

If LabConfig must be retained for internal tooling, the appropriate mitigation is to gate it behind a build-time flag that is absent from retail binaries. It should not be readable, writable, or effective on any system running a production Windows 11 build.

Recommendation 2: Fix Hardware Verification in Scenario 4 *(Immediate)*

The default installation path — Scenario 4 — routes through **EvaluateHardwareRequirementV2**, a reduced evaluation that checks for TPM presence but not TPM version. A system with TPM 1.2 passes this check without error. Windows 11 requires TPM 2.0.

This is not an edge case. Scenario 4 is the path every user takes when they boot from Windows 11 installation media and proceed normally. The strict evaluation path — **EvaluateHardwareRequirement** — exists and is already implemented. It is simply not called on the default installation path.

The fix is targeted: **AreHWRequirementsMet** should route Scenario 4 through **EvaluateHardwareRequirement**, the same strict evaluation applied to upgrade paths when no bypass key is present. The version check for TPM 2.0 should not be optional on any installation path for a Windows 11 product.

Recommendation 3: Audit AllowUpgradesWithUnsupportedTPMOrCPU as a Persistence Vector *(Immediate)*

AllowUpgradesWithUnsupportedTPMOrCPU was designed to allow systems with unsupported CPUs or older TPM hardware to receive Windows 11 feature upgrades. In isolation, this is a deliberate policy decision. In the context of the preceding findings, it becomes something else: the mechanism that makes a LabConfig-installed system permanent.

A system installed through LabConfig was never compliant with Windows 11's hardware requirements. Without an additional mechanism, it would be blocked from every subsequent feature upgrade. **AllowUpgradesWithUnsupportedTPMOrCPU** is that additional mechanism. Together, LabConfig handles installation-time bypass and this key handles upgrade-time bypass — a non-compliant system, once installed, has a supported path to remain on Windows 11 indefinitely.

The recommended action is to evaluate whether systems that cannot demonstrate compliance with Windows 11's baseline hardware requirements — Secure Boot, TPM 2.0 — should be eligible to receive feature upgrades at all, and whether **AllowUpgradesWithUnsupportedTPMOrCPU** should require a compliance gate before it takes effect.

Recommendation 4: Enforce Secure Boot Verification at Boot Time *(Long-Term)*

The three preceding recommendations address the installation-time conditions that produce non-compliant systems. This recommendation addresses what happens after those systems exist.

A system installed through LabConfig boots normally, receives updates, and operates indefinitely without any indication that its security posture is compromised. The kernel receives **BlVsmpSystemPolicy = 2**. HVCI, Credential Guard, and Kernel DMA Protection do not initialize. BitLocker cannot bind to the TPM. The user sees none of this.

bootmgfw.efi already queries the Secure Boot state during early boot through **DbxSvnIsSecureBootEnabled**. The infrastructure to detect a non-compliant boot environment is present. What is absent is any response to that detection beyond silently degrading the security posture.

A proportionate response does not require blocking boot. It requires transparency: a user who installed Windows 11 through a community bypass method should be informed — at boot time, clearly — that their system is not running with the security guarantees Windows 11 is designed to provide. This serves the user who made that choice unknowingly, and it creates a documented, auditable state that distinguishes intentionally non-compliant systems from systems that simply lack supported hardware.

The implementation path already exists in the boot stack. The policy decision is the remaining step.

A Note on Backward Compatibility

These recommendations will produce a predictable objection: implementing them would affect the large installed base of systems running Windows 11 through LabConfig or Rufus-based bypasses.

That objection is valid, and it is worth stating directly: closing these paths will have a real impact on real users. Some of those users made an informed choice. Many did not know what they were enabling.

The question this report is asking Microsoft to consider is not whether closing these paths is costless — it is whether the current state is acceptable. Windows 11 is publicly described as an operating system whose security guarantees are built into its hardware requirements. Every system running Windows 11 without Secure Boot or TPM 2.0 is a system where those guarantees do not hold, and where the user has no indication that this is the case.